Problem Set — Week 1

Formal Algorithm Analysis · VCE Algorithmics (HESS) · Unit 4 AOS1

Sections A–F correspond to Topics 1–6. Attempt each after its lesson.

Name: _____ **Due:** Tuesday 21 July

Method reminder: name the input size $n \to$ count how many times each loop runs $\to$ *price the body* $\to$ nesting multiplies, sequence takes the maximum $\to$ state the tightest bound *and* justify it.

Note: the Time Complexity Test (SAT Criterion 5) is **Friday 24 July**. Everything in this set is examinable.

Section A — Growth rates and Big-O *(Topic 1)*

A1. Order these from slowest-growing to fastest-growing:

$$n^{1.5}, \qquad \log n, \qquad n!, \qquad n \log n, \qquad 2^{\sqrt{n}}, \qquad 2^n, \qquad n, \qquad n^3$$

A2. Simplify each to its tightest Big-O form. One of these is a trap — say which and why.

a.) $7n^3 + 900n^2 + 40$

b.) $n^2 \log n + n^3$

c.) $5000 \log_2 n + 3000 \log_{10} n$

d.) $\frac{n(n+1)}{2} + 6n$

e.) $600 \times (0.4)^n$

f.) $2^n + n^{100}$

A3. Using the formal definition, show that $f(n) = 5n^2 + 3n + 8$ is $O(n^2)$ by exhibiting constants c and n_0 with $f(n) \leq c\,n^2$ for all $n \geq n_0$. Show the bounding of each term.

A4. An algorithm is $O(n^3)$ and processes $n = 400$ in 2 seconds. Estimate the time for $n = 4000$, and state one reason your estimate might be wrong in practice.

Section B — Θ, Ω, and which input *(Topic 2)*

B1. Define $\Theta(g(n))$ in terms of O and Ω, and explain in one sentence why Θ is a stronger claim than O.

B2. Consider linear search over a list of length n.

a.) State the best-case complexity and give a specific input ($n \geq 5$) that achieves it.

b.) State the worst-case complexity and give a specific input that achieves it.

c.) The average case, assuming the target is present and equally likely to be at any position, requires how many comparisons? Give the complexity class.

B3. Early-exit bubble sort is $O(n^2)$ and $\Omega(n)$. A student writes that it is "$\Theta(n^2)$". Explain precisely why that claim fails when quantifying over all inputs, and state what *can* correctly be written using Θ.

B4. An algorithm finds the maximum of an unsorted list of length n. A student claims its best case is $\Theta(1)$, "because the maximum might be the first element". Explain why this is wrong, and state a general principle about lower bounds on the best case.

Section C — Analysing iterative pseudocode *(Topic 3)*

For each algorithm, state the **worst-case** complexity in tightest form with a one-sentence justification. Let n be the length of the input list unless stated otherwise.

C1.

```
Algorithm Triangular(L):
    total ← 0
    Foreach i from 1 To n Do
        Foreach j from i To n Do
            total ← total + L[i] * L[j]
    Return total
```

C2. *Careful — price the body.* Here `Seen` is a **list**.

```
Algorithm HasDuplicate(L):
    Seen ← empty list
    Foreach x in L Do
        If x in Seen Do
            Return True
        Append x to Seen
    Return False
```

a.) State the complexity and justify it.

b.) State the complexity if `Seen` were a dictionary instead, and explain the difference.

C3.

```
Algorithm Mixed(L):
    i ← n
    While i > 1 Do
        Foreach x in L Do
            print(x)
        i ← i div 3
    Return
```

C4. A student states that the algorithm below is $O(n)$ "because there is one loop". Give the correct complexity and identify precisely what the student overlooked.

```
Algorithm Repeated(L):
    Foreach x in L Do
        Sort(L)
    Return L
```

Section D — Graph algorithm complexity *(Topic 4)*

D1. Explain in two sentences why breadth-first search is $O(|V| + |E|)$ and **not** $O(|V| \times |E|)$. Your answer must refer to how many times each edge is examined across the whole run.

D2. Complete the table, giving the data structure each bound assumes.

Algorithm	Complexity	Structure assumed
BFS / DFS		
Dijkstra (heap)		
Dijkstra (array)		
Prim's		
Floyd–Warshall		

D3. On a *dense* graph, which implementation of Dijkstra is faster — binary heap or array? Justify by substituting an appropriate expression for $|E|$ into both bounds.

D4. An algorithm runs a full BFS starting from every node of a graph. State its complexity and explain why the edge walks do **not** sum to $O(|E|)$ in this case.

D5. Explain why Floyd–Warshall is quoted as $\Theta(|V|^3)$ rather than $O(|V|^3)$.

Section E — Call graphs and recursion *(Topic 5)*

E1. For each algorithm, write the recurrence relation *including its base case*.

a.) Binary search on a sorted list of length n.

b.) Merge sort on a list of length n.

c.) An algorithm making 4 recursive calls on inputs of size $n/2$, with $O(n^2)$ work outside the calls.

E2. Sketch the call graph of `F(5)` for the algorithm below, and state how many times `F(2)` is evaluated.

```
Algorithm F(n):
    If n   1 Do
        Return 1
    Return F(n-1) + F(n-2)
```

E3. Explain why naive Fibonacci is exponential while merge sort is $\Theta(n \log n)$, given that *both* make two recursive calls per invocation. Your answer must refer to the depth of the recursion.

E4. *Exam-style.* A machine sorts a bag of n items by reading each item once (constant time per item) and distributing them into four bags of roughly equal size, then repeating the process on each bag. Let $S(n)$ be the time complexity. Explain why $S(n) = 4S(n/4) + O(n)$, accounting for *each* of the three components separately.

Section F — Recurrences and the Master Theorem *(Topic 6)*

The Master Theorem is supplied in the exam. For $T(n) = a\,T(n/b) + kn^c$, compare $\log_b a$ with c.

F1. Solve each using the Master Theorem. State a, b, c and $\log_b a$ in every case.

a.) $T(n) = 2T(n/2) + n$

b.) $T(n) = 8T(n/2) + n^2$

c.) $T(n) = 9T(n/3) + n^2$

d.) $T(n) = 3T(n/4) + n$

e.) $T(n) = 2T(n/2) + 1$

f.) $T(n) = T(n/2) + 1$

F2. Solve $T(n) = T(n-3) + n$ by expansion. Set out the five moves: substitute, substitute again, generalise to k, choose k to reach the base case, substitute back.

F3. A student attempts to apply the Master Theorem to $T(n) = 2T(n-1) + 1$ and obtains $O(n)$. Explain why the Master Theorem cannot be used here, and give the correct complexity with brief justification.

F4. Without solving them fully, predict the complexity class of each from its shape alone, and say which of the two methods you would use.

a.) $T(n) = T(n/2) + O(n)$

b.) $T(n) = T(n-1) + O(1)$

c.) $T(n) = 2T(n/2) + O(n^2)$

d.) $T(n) = T(n-2) + O(n)$

Section G — Connecting to your SAT

G1. Choose one module of your Unit 3 solution that contains a loop over your graph's nodes or edges. Write down its loop structure (paraphrase is fine), identify what n means for it, price the body, and give a first estimate of its worst-case complexity.

This is the module inventory that Memo 02 will ask for — start it now.

Bring your attempt to the first lesson of Week 2. Sections C and D will be marked together before the Time Complexity Test on Friday 24 July.